Ayame: Fast Logging for Serial Safety Net

Shun Sasaki[1,a)] Takayuki Tanabe[1,b)] Hideyuki Kawashima[2,c)]

Abstract: Concurrency control is essential for database access, and the Serial Safety Net (SSN) represents one of the state-of-the-art methods in this field. Integrating SSN with logging ensures that transaction processing is recoverable. While P-WAL is an efficient logging method that leverages parallel I/O extensively, it still suffers from certain limitations. To overcome these drawbacks, this paper proposes a novel logging method, Ayame. The proposed method incorporates three optimization techniques: the elimination of the separate WAL-side global LSN allocation, dependency analysis, and an asynchronous durable-acknowledgment pipeline with background batched flushing. Experimental results demonstrate that Ayame achieves a 4.1- to 6.5-fold performance improvement over P-WAL in YCSB-A.

1. Introduction

1.1 Motivation

Transaction processing is a core abstraction behind data-intensive applications such as banking, payment processing, inventory management, ticket reservation, and online services that update shared state under high concurrency. A transaction processing system must provide both correct concurrent execution and crash recovery; in this sense, transaction processing consists of concurrency control (CC) and recovery [1]. A large body of work has studied scalable CC for multicore main-memory databases, including Silo, ERMIA, TicToc, SI, SSI, and SSN-style protocols [2], [3], [4], [5], [6], [7]. These protocols improve isolation and scalability by reducing centralized validation, timestamp, and version-management bottlenecks. In contrast, the recovery side of modern CC protocols has been explored less systematically: when a high-performance CC protocol is combined with crash durability, it is often unclear which recovery protocol preserves both safety and scalability.

Write-ahead logging (WAL) is the standard foundation for database recovery, with ARIES being the classical design [8], [9]. Modern systems have revisited WAL for multicore and fast-storage environments. SiloR parallelizes logging, checkpointing, and recovery for Silo; Passive Group Commit reduces synchronization overhead on non-volatile memory; and P-WAL partitions the log into multiple streams [10], [11], [12]. Although these systems differ in the target storage device, log format, and recovery procedure, they share an important lesson: writing log records in parallel is essential for making durability compatible with multicore transaction processing. At the same time, strong multiversion protocols such as SI, SSI, and SSN introduce timestamp and dependency structures that are not fully reflected in conventional parallel logging designs. The motivation of this work is therefore to clarify how P-WAL-style parallel logging should be combined with such timestamp-based CC protocols, especially SSN, without unnecessarily reintroducing global ordering and durable-prefix bottlenecks.

1.2 Problem

We target SSN because it is the most complex of the SI-family protocols we consider: it certifies serializability using dependency metadata, while the underlying engine still uses timestamped multiversion execution. As the baseline recovery protocol, we start from P-WAL, which is also used as a foundation of Shirakami, the transaction processing mechanism of Tsurugi [12], [13]. P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-

[1] Graduate School of Media and Governance, Keio University, Japan
[2] Faculty of Environment and Information Studies, Keio University, Japan
[a)] shun.sasaki@keio.jp
[b)] tanab@keio.jp
[c)] river@sfc.keio.ac.jp

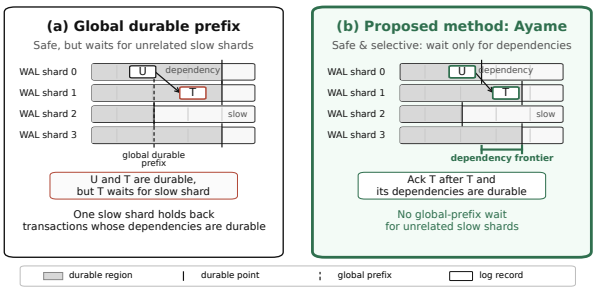

Fig. 1 Durable acknowledgment policies in parallel WAL. Global-prefix is safe but waits for unrelated slow shards; local-only avoids the wait but is unsafe; Ayame's dependency-closed rule waits only for a transaction's own record and its dependency frontier.

prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

P3. Unrelated-shard waiting under global-prefix acknowledgment: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

In asynchronous durability protocols, logical commits and durable commit acknowledgments must be distinguished. A worker may continue after enqueueing a log record, but the transaction is not durable from the client's perspective until the system returns a durable acknowledgment. Therefore, this paper uses *durable-acknowledgment throughput* as the main throughput metric and reports it together with pending durable commits and tail latency.

1.3 Approach

This paper proposes *Ayame*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. Ayame targets timestamp-based engines such as ERMIA, where the transaction engine already computes a logical commit order. Its goal is not merely to maximize raw logical commit throughput, but to return durable commit acknowledgments safely without unnecessary global-prefix waiting.

Ayame addresses the three problems above with three corresponding techniques.

A1. Commit timestamps as logical LSNs. Ayame reuses the SSN/ERMIA commit timestamp as the logical LSN for recovery. WAL shards still maintain local physical positions, but the WAL layer does not allocate an additional global logical order. This removes duplicated global ordering between CC and recovery and eliminates WAL-side separate global LSN allocation on the commit path.

A2. Worker/flusher/committer separation. Ayame decouples transaction execution from durable acknowledgment. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local logs. Committers observe durable-frontier advances and return durable acknowledgments. This pipeline keeps workers from directly blocking on `fdatasync` or on durable-prefix notification waits.

A3. Dependency-closed durable acknowledgment. Ayame replaces global-prefix acknowledgment with dependency-closed acknowledgment. For each transaction, Ayame maintains a dependency frontier that summarizes the log positions that must be recoverable before the transaction can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This preserves recovery safety while avoiding waits for unrelated WAL shards.

We implement Ayame in an ERMIA-style engine and evaluate it against P-WAL using SSN and YCSB workloads. The detailed experimental methodology, figures, and tables are presented in Section 4.

1.4 Organization

The rest of this paper is organized as follows. Section 2 describes transaction processing, including serial-safety-net and parallel logging protocols. Section 3 presents Ayame, which is a fast logging protocol. Section 4 describes the evaluation of Ayame compared with the P-WAL protocol using SSN. Section 5 discusses related work. Section 6 finally concludes this paper.

2. Preliminaries

2.1 Concurrency Control

A database transaction is a unit of execution that must satisfy atomicity, consistency, isolation, and durability [8]. Concurrency control (CC) is responsible for isolation: it interleaves the operations of concurrent transactions so that the outcome is equivalent to some serial execution, the standard correctness criterion of serializability [1]. CC protocols differ in how they detect and resolve the read–write, write–read, and write–write conflicts that would otherwise violate serializability. Pessimistic protocols such as two-phase locking block conflicting operations as they occur, whereas optimistic protocols let transactions execute first and validate them at commit time, aborting those whose interleaving cannot be serialized. Main-memory multicore databases favor optimistic and multiversion designs because they avoid the lock-manager and latch contention that limit scalability on many cores [2], [3], [7].

2.2 Timestamp-based Protocols

Multiversion concurrency control retains several versions of each record so that reads need not block writes. Snapshot isolation (SI) runs each transaction against a consistent snapshot taken at its start and, at commit, checks only for write–write conflicts; it is efficient but admits write-skew anomalies and is therefore not serializable. Serializable Snapshot Isolation (SSI) strengthens SI by detecting the dangerous read–write antidependency structures that can lead to non-serializable schedules [5]. These protocols order transactions with timestamps: a transaction reads as of a begin timestamp and commits at a commit timestamp, and the commit timestamps induce a total order that defines the serialization order. Because these commit timestamps already encode the serialization order, they can also serve as the logical order in which a recovery protocol replays committed transactions after a crash.

2.2.1 Serial Safety Net

The Serial Safety Net (SSN) is a certifier layered on top of a multiversion engine such as SI or ERMIA to guarantee serializability [3], [6]. Rather than maintaining the full dependency graph, SSN summarizes, for each transaction T, the commit timestamps of its predecessors and successors in the dependency graph into a pair of watermarks, and it refuses to commit T when these watermarks reveal that committing T would close a non-serializable cycle. Each committing transaction receives a commit timestamp (*cstamp*) that fixes its position in the serialization order, and the read and write footprints of in-flight and recently committed transactions are tracked through a shared transaction-mapping table so that validation can read the relevant predecessors' and successors' timestamps. SSN admits more concurrency than two-phase locking while remaining serializable, which makes it a strong but demanding target for a recovery protocol. The recovery layer must respect the same commit-timestamp order that SSN uses for serialization, yet it must do so without introducing new centralized bottlenecks of its own. The shared commit-timestamp counter and transaction-mapping table are also the structures whose cache-line contention ultimately bounds throughput at high core counts, a known scalability limit of SSN-style certification [7].

2.3 Write-Ahead Logging for Recovery

Durability and atomicity across crashes are provided by write-ahead logging (WAL). Under the WAL rule, a transaction's log records are forced to stable storage before the transaction is reported as committed, so that a crash can be recovered by replaying the log. ARIES is the classical WAL design: each log record carries a monotonically increasing log sequence number (LSN), recovery repeats history in a redo pass and then rolls back loser transactions in an undo pass, and fuzzy checkpoints bound the amount of log that must be scanned [8], [9]. Forcing the log on every commit is costly because it requires a storage synchronization (`fdatasync`) on the commit path; group commit amortizes this cost by making a single synchronization durable for a batch of commits. In a main-memory database the redo log is the dominant persistence cost, so the rate at which log records can be made durable—and the latency of doing so—directly bounds commit throughput.

Algorithm 1 Per-Thread WAL Commit for Update Transactions (P-WAL)

```
1: Shared: global LSN counter L; per-stream durable LSN f[i]; global durable prefix P = min_i f[i]
2: function PwalCommit(T, worker w)
3:     records ← BuildLogRecords(T)        ▷ writes and commit record
4:     for all r ∈ records do
5:         r.lsn ← FetchInc(L)             ▷ global LSN allocation
6:         Append(stream[w], r)            ▷ worker-private WAL stream
7:     end for
8:     T.ℓ ← max{ r.lsn | r ∈ records }    ▷ commit LSN
9:     Write(stream[w]); Fdatasync(stream[w])   ▷ worker waits for persistence
10:    f[w] ← max(f[w], T.ℓ)
11:    P ← min_i f[i]                      ▷ global durable prefix
12:    while P < T.ℓ do
13:        P ← min_i f[i]
14:    end while
15:    Acknowledge(T)
16: end function
```

2.4 Parallel Logging

A single shared log serializes every commit through one log tail, which becomes a bottleneck on many cores. Parallel WAL designs remove this bottleneck by partitioning the log into multiple streams that are appended and flushed independently [10], [11], [12].

2.4.1 P-WAL

P-WAL gives each worker its own log stream, which we hereafter call a *WAL shard*, so that workers append commit and redo records without contending on a shared log tail [12]. Algorithm 1 shows the resulting commit path. Because the shards are independent, P-WAL re-establishes a single global order across them: every record is stamped with a value drawn from a global LSN counter L, and each shard publishes a durable position $f[i]$ whose minimum over all shards forms a global durable prefix $P = \min_i f[i]$. A transaction is acknowledged once the global prefix reaches its commit LSN, which guarantees that every earlier record, on every shard, is already durable. P-WAL underlies Shirakami, the transaction engine of Tsurugi [12], [13].

This design makes log insertion parallel, but combining it with a timestamp-based certifier such as SSN raises the three problems (P1–P3) detailed in Section 1.2.

3. Proposal: Ayame

Ayame is a logging method that integrates durable write-ahead logging into SSN while avoiding the three problems of P-WAL identified in Section 1.2. It combines three techniques. First, it reuses the commit timestamp that SSN already assigns as the logical log sequence number, eliminat-

ing the separate WAL-side global counter (addresses **P1**). Second, it splits commit processing into worker, flusher, and committer roles so that worker threads do not block on `fdatasync` or durable-ack waiting, subject only to a bounded backpressure limit (addresses **P2**). Third, it acknowledges a transaction using a *dependency-closed durable frontier* instead of a global durable prefix, so that a transaction waits only for the log records it actually depends on (addresses **P3**).

We use the following notation. A frontier F is a vector of per-shard log positions. $F \sqcup F'$ denotes the element-wise maximum, $F \leq F'$ holds iff $F[i] \leq F'[i]$ for every shard i, and $\perp$ denotes the empty frontier whose positions are all zero. D denotes the durable vector, where $D[i]$ is the largest log position of shard i that is known to be durable; each $D[i]$ advances monotonically.

3.1 Commit Timestamp as Logical LSN

In SSN, a committing transaction T is assigned a commit timestamp $T.c$ (its *cstamp*) that fixes its position in the serialization order. We assume that SSN assigns a unique commit timestamp to each committed update transaction, so these timestamps form a total order that recovery uses as the replay order. A separate WAL-side global LSN, as used by P-WAL, would redundantly re-encode the same logical order and add a second global-counter access to every commit. Ayame instead reuses $T.c$ directly as the logical LSN that orders recovery replay. No WAL-side global sequence number is allocated; in our evaluation the per-transaction WAL-side global-counter access drops to zero.

Removing the global *logical* LSN does not remove physical log addressing. Each WAL shard keeps a local position (a per-shard sequence number and byte offset) that locates a record within its log file. What Ayame eliminates is the duplicated global ordering counter, not the per-shard physical position.

3.2 Worker, Flusher, and Committer Separation

Ayame divides commit processing into three roles. A *worker* executes the transaction, validates it with SSN, assigns the commit timestamp, appends the log record to its own WAL shard, publishes durability metadata, and registers a durable-acknowledgment request. The worker then returns to its next transaction; it does not call `fdatasync` and does not wait for a durable acknowledgment. A *flusher* owns one WAL shard, batches the queued records, writes them, calls `fdatasync`, and advances the shard's durable position. A *committer* returns durable acknowledgments to clients once the durable condition of a registered transaction is met. Workers therefore do not block on storage synchronization or durable-ack waiting, and keep executing transactions at full concurrency even though every update commit must still be made durable; the cost of making the log durable is paid by the flusher and committer. The one exception is a bounded backpressure limit: to cap memory use and acknowledgment latency, a worker stalls only if the number of registered but not-yet-acknowledged durable commits reaches a configured maximum. Accordingly, we report throughput as the number of durable acknowledgments returned to clients, not the number of logical commits.

Algorithm 2 Worker-Side Commit Protocol (Ayame)

```
1: function COMMIT(T)
2:     T.c ← commit timestamp (cstamp) assigned by SSN ▷ logical LSN
3:     if SSNVALIDATE(T) fails then
4:         abort T; return
5:     end if
6:     if T's write set is empty then ▷ read-only
7:         ACK(T) ▷ read-only has no recoverable effect
8:         return logical read-only commit
9:     end if
10:    s ← WAL shard assigned to this worker
11:    T.seq ← RESERVESEQ(wal[s]) ▷ reserve this transaction's log position
12:    F ← T.dep;  F[s] ← max(F[s], T.seq) ▷ closed frontier
13:    r ← BUILDRECORD(T.c, F, T.write_set) ▷ commit record carries F
14:    ENQUEUE(wal[s], r) ▷ append only; no flush, no wait
15:    for all versions v created by T do
16:        v.wf ← F
17:    end for
18:    PUBLISHVERSIONS(T.write_set) ▷ visible only after wf is installed
19:    for all versions v read by T do
20:        v.rf ← v.rf ⊔ F ▷ monotone merge
21:    end for
22:    REGISTER(F, T) ▷ durable ack deferred to the committer
23:    return after logical commit registration
24: end function
```

Algorithm 2 shows the worker-side commit protocol. The commit timestamp $T.c$ assigned for SSN validation doubles as the logical LSN (line 2). After validation, the worker reserves its log position on its shard (line 11) and forms the closed frontier F by extending its dependency frontier $T.dep$ with that position (line 12). It then builds the log record, whose commit record carries F (line 13), and enqueues it without flushing or waiting (line 14). Ayame installs F as the write frontier of the versions T created (line 16) and publishes those versions (line 18); a version becomes visible only after its write frontier is installed, so any later reader or overwriter observes the writer's complete frontier. It also merges F into the read frontiers of the versions T read (line 20). Finally, the worker registers F for durable acknowledgment (line 22) and returns to the execution scheduler; the client-visible commit response is returned only later by the committer, once the frontier condition is satisfied.

The flusher does not require an algorithm of its own. Each flusher owns one WAL shard, repeatedly collects the records queued by workers, writes them as a single batch, calls `fdatasync` once for the batch, and then advances the shard's durable position by invoking ONDURABLEADVANCE (Algorithm 4). Batching many records per `fdatasync` is what raises the number of commits made durable per syn-

Algorithm 3 Dependency Collection (Ayame)

```
 1: function Begin(T)
 2:     T.dep ← ⊥
 3: end function

 4: function ReadVersion(T, v)
 5:     add v to T's read set
 6:     if T is known read-only then
 7:         return                          ▷ SI read-only fast path
 8:     end if
 9:     T.dep ← T.dep ⊔ v.wf                ▷ the writer of v
10: end function

11: function OverwriteVersion(T, v)
12:     add the new version to T's write set
13:     T.dep ← T.dep ⊔ v.wf ⊔ v.rf        ▷ the writer of v and its
    readers
14: end function
```

Algorithm 4 Durable Acknowledgment (Ayame)

```
    State: durable vector D; per-shard waitlists W[i] ordered by
    required position
 1: function Register(F, T)                 ▷ atomic w.r.t.
    OnDurableAdvance
 2:     T.remaining ← 0
 3:     for all shards i such that F[i] > D[i] do
 4:         T.remaining ← T.remaining + 1
 5:         insert (F[i], T) into W[i]
 6:     end for
 7:     if T.remaining = 0 then
 8:         Ack(T)      ▷ own record and all dependencies durable
 9:     end if
10: end function

11: function OnDurableAdvance(i, d)         ▷ called by flusher i
    after fdatasync
12:     D[i] ← d
13:     while W[i] is not empty and MinKey(W[i]) ≤ d do
14:         (need, T) ← PopMin(W[i])
15:         T.remaining ← T.remaining − 1
16:         if T.remaining = 0 then
17:             Ack(T)
18:         end if
19:     end while
20: end function
```

chronization and reduces flush pressure relative to P-WAL's per-transaction synchronization.

3.3 Dependency-Closed Durable Acknowledgment

The remaining question is when a logically committed transaction may be acknowledged as durable. Ayame answers this with a frontier attached to each version. A version carries a *write frontier* wf, the closed frontier of the transaction that created it. A writer publishes wf before its version becomes visible, so any transaction that later reads or overwrites the version observes the complete write frontier of its producer. During execution, a transaction joins these write frontiers into its dependency frontier $T.dep$, as shown in Algorithm 3: reading a version makes T depend on the writer of that version (line 9), and overwriting a version makes T depend on the writer of the version it supersedes (line 13). A version also carries a *read frontier* rf, into which readers monotonically merge their closed frontiers; OverwriteVersion additionally joins rf so that an overwriter conservatively tracks the readers of the version it supersedes. This read-frontier term is a conservative delay mechanism; it is not required for the recoverable-acknowledgment proof, which relies only on write frontiers published before a version becomes visible. Transactions known to be read-only skip WAL generation and durable-acknowledgment registration. When the read-only fast path applies, they also avoid dependency-frontier collection; otherwise, only lightweight read-path frontier bookkeeping may remain.

At commit, the closed frontier F (line 12 of Algorithm 2) extends $T.dep$ with T's own log position. By induction over the dependency order, F therefore contains the log positions of the transitive closure of the read-from and version-order (write-write) dependencies of T, all of which are recorded through write frontiers. Publishing F as the write frontier of the versions written by T lets future readers and overwriters inherit T's closed frontier through Algorithm 3. Merging F into the read frontier of the versions read by T conservatively delays future overwriters of those versions, but this read-frontier term is not required for the safety proof.

Algorithm 4 shows the committer side. Register parks T on each shard whose durable position has not yet reached the requirement of F (line 5); if no shard is behind, T is acknowledged immediately (line 8). OnDurableAdvance, invoked by a flusher after `fdatasync`, advances the durable position of one shard and wakes the parked transactions whose requirement on that shard is now satisfied, releasing the acknowledgment once a transaction's last outstanding shard becomes durable (line 17). Register and OnDurableAdvance execute under a single lock so that a durable advance cannot slip between the durability test and the parking step.

The acknowledgment rule is exactly $F \le D$: a transaction is acknowledged once its own record and every record it transitively depends on are durable. We now make the safety of this rule precise. For a committed update transaction U, let s_U and $U.seq$ be the shard and position of its log record, so its published closed frontier F_U satisfies $F_U[s_U] \ge U.seq$. Let $\mathrm{cl}(T)$ be the set of update transactions reachable from T through read-from dependencies (T reads a version written by U) and version-order dependencies (T overwrites a version written by U), including T itself. Both kinds of dependency are recorded by joining the relevant version's write frontier, which its writer publishes before the version becomes visible.

Lemma 1 (Dependency-closed durability) If Ack(T) is invoked, then for every update transaction $U \in \mathrm{cl}(T)$ the WAL record of U is durable.

Proof. Dependency collection (Algorithm 3) joins the write

frontier of each version T reads or overwrites into $T.dep$, and the commit protocol sets $F = T.dep$ with $F[s_T] \geq T.seq$ (line 12). A writer publishes its write frontier before its version becomes visible, so T observes the complete frontier F_U of every U it directly depends on; since each F_U is itself closed, induction over the dependency order gives $F \geq F_U$, hence $F[s_U] \geq F_U[s_U] \geq U.seq$, for every $U \in \mathrm{cl}(T)$. Ack(T) is invoked only when $F \leq D$ (Algorithm 4), so $D[s_U] \geq U.seq$. Since $D[i]$ is a durable prefix of shard i, the record of U is durable.

Each commit record stores the transaction's commit timestamp $T.c$ and its closed dependency frontier F, alongside the redo records that carry the write set. During recovery, Ayame reconstructs the durable vector D by parsing each shard's records in order and advancing $D[i]$ until the first record that does not parse as a complete, well-formed entry, which indicates a torn final write; records beyond that point are ignored. A distinguished commit marker identifies each transaction's commit record. Ayame then collects the committed transactions whose commit records lie within D, keeps only those whose stored frontier is covered by the durable vector (that is, $F \leq D$), and replays this dependency-closed set in increasing cstamp order. Because the replayed set is closed under $F \leq D$, a recovered transaction never appears without the transactions it depends on, even if its own commit record happened to be durable while a dependency's was not.

Theorem 1 (Recoverable acknowledgment) Suppose a crash occurs and recovery replays exactly the transactions whose stored frontier is covered by the durable vector ($F \leq D$), in increasing cstamp order. If T was acknowledged before the crash, recovery never yields a state that contains the effects of T but omits the effects of some $U \in \mathrm{cl}(T)$.

Proof. By Lemma 1, every $U \in \mathrm{cl}(T)$ has a durable record and is therefore replayed. For the dependency classes included in $\mathrm{cl}(T)$—read-from dependencies and version-order dependencies from overwritten versions—the producer version must have become visible before T reads or overwrites it, so under the engine's MVCC visibility rule the producer's commit timestamp precedes T's. Because logical LSNs equal commit timestamps, replaying the acknowledged transactions in increasing cstamp order installs every such U before T. Hence whenever T's effects are installed, the effects of all $U \in \mathrm{cl}(T)$ are already present.

The lemma and theorem rest only on write frontiers, which are published before a version is visible and are therefore observed completely by every reader and overwriter. The read-frontier term that OverwriteVersion joins (the *rf* of an overwritten version) makes acknowledgment additionally wait for the readers of that version, an anti-dependency. This term is a conservative overapproximation: it can only delay an acknowledgment, never release one early, and the safety argument above does not use it. Because readers publish *rf* concurrently with overwriters, it is a best-effort addition rather than a guaranteed bound, which is why we keep the proof over write frontiers alone.

The rule is also less conservative than a global durable prefix: a transaction waits only on the shards that appear in its frontier, so an unrelated slow shard never delays it.

4. Evaluation

4.1 Implementation

We implement Single WAL, P-WAL, and Ayame on CCBench [7], a concurrency-control benchmarking framework with minimal functionality for protocol analysis, using Masstree as the index and `mimalloc` as the allocator. The code is written in C++20 and compiled with GCC at the `-O3` optimization level. Our implementation is publicly available.[*1]

A key methodological point is that Single WAL, P-WAL, and Ayame are implemented in the *same* binary and selected at run time by environment flags (`CCBENCH_WAL_MODE` and `CCBENCH_WAL_DURABLE_MODE`). The CCBench SI+SSN protocol implementation, the index, the workload generator, and the pre-commit path are therefore identical across the three systems; only the WAL durability path differs. Single WAL appends to one shared log under a global latch and synchronizes it per commit. P-WAL gives each worker a private WAL shard and synchronizes it per commit, acknowledging through a global durable prefix. Ayame uses the worker/flusher/committer pipeline of Section 3 with dependency-closed acknowledgment. Each WAL redo record carries the key, value, storage identifier, and operation type of a write; the terminating commit record additionally stores the transaction's commit timestamp and its closed dependency frontier, which recovery uses to admit only the dependency-closed set of durable transactions. Read-only WAL skipping is enabled for all three systems; once a transaction is known to be read-only it produces no WAL record. For read-only transactions Ayame also skips durable-acknowledgment registration, but depending on the read path it may still execute lightweight frontier bookkeeping; YCSB-C quantifies this residual overhead.

Following the worker/flusher/committer separation of Section 3, Ayame offloads durability to background threads: it uses 1, 3, 5, 7, 9, 12, 14, 17, and 19 flusher threads for 1, 12, 24, 36, 48, 60, 72, 84, and 96 worker threads, respectively, and one committer thread in all configurations. The x-axis of our worker-scaling experiments is the number of transaction worker threads; at 48 workers Ayame therefore runs 58 active threads (48 workers, 9 flushers, and one committer). At 96 workers Ayame runs 116 active threads (96 workers, 19 flushers, and one committer), which exceeds the 96 logical cores of the machine; we revisit this oversubscription when discussing the high-thread end of the sweep.

[*1] The source code is available at `https://github.com/sa2shun/ccbench-wal/tree/cstamp-pwal-framework`.

Table 1 Default parameter settings.

Type	Parameter	Setting
DB	#records	100,000
	value size	4 B
	key distribution	uniform
TX	#operations	10
	read ratio	50 / 95 / 100 %
Run	worker threads	1,12,24,36,48,60,72,84,96
	measurement time	5 s
	repeats	5
Ayame	flush group size [commits]	256
	flush interval	50 μs
	max pending acks	65,536

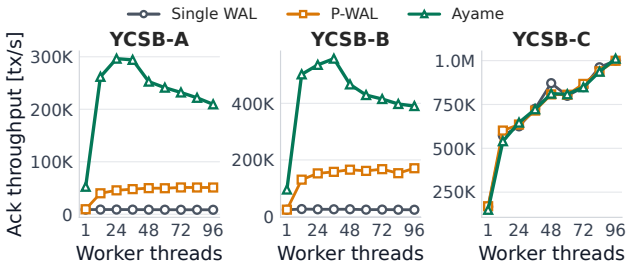

Fig. 2 Durable-acknowledgment throughput by transaction worker threads. Ayame improves update and mixed read-write workloads by combining parallel WAL logging, dependency-closed durable acknowledgment, and timestamp-integrated logical ordering.

4.2 Experimental Configuration

4.2.1 Environment

Experiments were conducted on a server with two sockets of Intel(R) Xeon(R) Gold 5418N CPUs. Each socket had 24 physical cores with two hardware threads per core, for 48 physical and 96 logical cores in total, organized as two NUMA nodes. Each core had a 48 KiB private L1d cache and a 2 MiB private L2 cache, and each socket shared a 45 MiB L3 cache. The DRAM capacity was 256 GB. The machine ran Ubuntu 22.04.5 LTS (Linux kernel 5.15), and WAL files were placed on a locally attached SSD formatted with ext4. Worker threads were pinned to distinct logical cores with `sched_setaffinity`.

4.2.2 Benchmark and Workloads

All experiments used the Yahoo! Cloud Serving Benchmark (YCSB) [14]. We evaluated three representative workloads: YCSB-A (50% read, 50% update), YCSB-B (95% read, 5% update), and YCSB-C (read-only). These workloads exercise the WAL along a spectrum from update-heavy to read-only and are widely used to evaluate concurrency-control protocols. Each transaction issued a fixed number of ten operations over a database of 100,000 records with 4-byte values, and keys were drawn from a uniform distribution.

4.2.3 Metric

The primary metric is *durable-acknowledgment throughput*: the number of transactions acknowledged as durable to the client, divided by the measurement time. All systems are measured at the same stop barrier, and acknowledgments drained during shutdown are excluded. For the worker-wait systems (Single WAL and P-WAL) a logical commit and a durable acknowledgment coincide, so the metric has the same meaning across the three systems. We additionally report the p99 durable acknowledgment latency and the number of pending (not-yet-acknowledged) durable commits. Unless otherwise stated, each reported value is the mean of the corresponding per-run metric over five runs; reported p99 latencies are means of the per-run p99 values.

4.3 Results

We evaluate Ayame using the worker-thread sweep of Table 1, which adds a single-thread baseline and then spans 12 to 96 worker threads in steps of 12 (up to the full two-socket machine). At 48 worker threads on YCSB-A, Single WAL achieves 9K durable acknowledgments per second and P-WAL achieves 50K, whereas Ayame achieves 253K durable acknowledgments per second. On YCSB-B, Single WAL achieves 26K and P-WAL achieves 165K, whereas Ayame achieves 467K durable acknowledgments per second. These results correspond to 5.1× and 2.8× improvements over P-WAL on YCSB-A and YCSB-B, respectively. Ayame also keeps the number of pending durable commits small at 48 worker threads—318 on YCSB-A and 83 on YCSB-B—with a 2 ms p99 acknowledgment latency on YCSB-A and 0.5 ms on YCSB-B. Figure 4 shows that the p99 durable-acknowledgment latency stays low across the worker sweep. This end-to-end comparison evaluates Ayame as a complete durability pipeline; it should not be read as isolating the effect of dependency-closed acknowledgment alone, since the throughput gain over P-WAL mainly reflects worker/flusher/committer separation and batched `fdatasync` (Section 4.4). Ayame's YCSB-A throughput peaks near 24–36 workers (294–296K) and then declines gradually toward 96 workers (209K), and YCSB-B follows the same shape. P-WAL declines in parallel, so the Ayame-over-P-WAL ratio stays large throughout, ranging from 6.5× at 12 workers to 4.1× at 96. We analyze the cause of this peak-and-decline below (Figure 3).

Figure 2 shows the worker-thread comparison. The results show that Ayame improves update and mixed read-write workloads, where durable logging is still required on every update commit. We also evaluate YCSB-C, where all transactions are read-only and WAL records are skipped. In this case, durability work largely disappears; with the same binary and a read-only empty-frontier fast path, the three systems land within about 8% of one another at 48 worker threads (872K for Single WAL, 808K for P-WAL, and 812K for Ayame in Table 2). Ayame is marginally below Single WAL because it still maintains read-path frontier bookkeeping (72 bytes per transaction). We therefore use YCSB-C as a sanity check for read-only bookkeeping overhead rather than as evidence of WAL persistence performance.

This peak-and-decline is a property of the underlying SSN concurrency control rather than of Ayame's WAL pipeline, and it is not caused by aborts. Figure 3 reports the per-committed-transaction CPU cost and the abort rate across

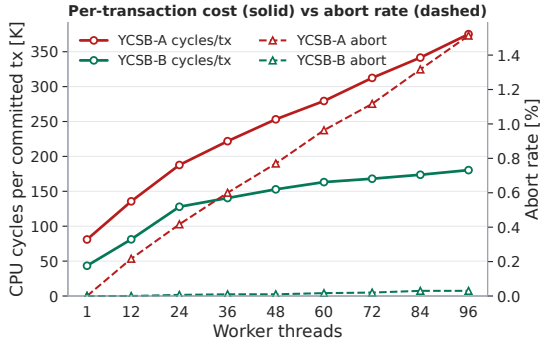

Fig. 3 Per-committed-transaction CPU cost (solid, left axis) and abort rate (dashed, right axis) versus worker threads for Ayame. The cycles per committed transaction rise sharply with thread count—cache-line contention on SSN's central commit-timestamp counter and transaction-mapping table—while the abort rate stays small; YCSB-B declines with a near-zero abort rate.

the sweep. The cycles spent per committed transaction rise steeply with thread count—about 2.8× on YCSB-A (136K at 12 workers to 375K at 96) and 2.2× on YCSB-B—the classic signature of cache-line contention on shared structures. Because the per-transaction cost grows faster than the core count, throughput falls once the contention outweighs the added parallelism. The abort rate cannot explain the decline: it never exceeds 1.5% on YCSB-A and 0.03% on YCSB-B, yet YCSB-B exhibits the same peak-and-decline shape with that near-zero abort rate. The contended structures are central to SSN: every commit increments a single global commit-timestamp counter—which Ayame reuses as the logical LSN (**A1**)—and SSN validation consults a central transaction-mapping table, both of which bounce between cores' caches as the thread count grows. This is the known scalability limitation of SSN's centralized metadata [7]. The decline is not thread oversubscription: P-WAL carries no background flush threads, so at 96 workers it runs 96 threads on the 96 logical cores without oversubscribing them, yet its throughput falls in parallel with Ayame's—so the falloff reflects the shared SSN structures, not an excess of threads. The limitation is also upstream of durability: the p99 acknowledgment latency stays at or below 2 ms across the whole sweep (Figure 4) and the pending durable commits remain small at the operating point (Table 2), so the flushers and committer are not the bottleneck—the throughput ceiling is set by the concurrency-control layer that Ayame inherits through cstamp reuse.

Ayame's timestamp integration is primarily a design simplification: it removes duplicated logical ordering between concurrency control and WAL durability by reusing SSN's commit timestamp (cstamp) as the logical recovery order. We do not rely on timestamp integration alone as a direct throughput optimization. The main throughput effect comes from parallel WAL shards, batching, and separating workers from durable-wait processing; dependency-closed acknowledgment primarily prevents the pending-ack backlog and tail-latency blowups caused by global-prefix waiting (Section 4.4).

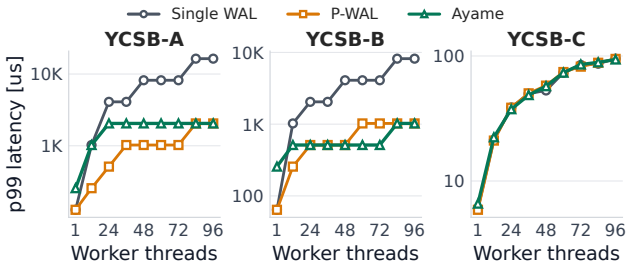

Fig. 4 P99 durable-acknowledgment latency by transaction worker threads.

Table 2 summarizes the main 48-worker end-to-end results used in the figures.

4.4 Acknowledgment-Policy Comparison

The main end-to-end comparison combines several changes: timestamp-integrated ordering, asynchronous flushing, and dependency-closed acknowledgment. To isolate the effect of the acknowledgment rule itself, we run an acknowledgment-policy comparison that uses the same worker/flusher/committer pipeline as Ayame and changes only the condition for returning durable acknowledgments. The *global-prefix* variant acknowledges a transaction only after a single global durable prefix covers its logical commit position; the *dependency-frontier* variant acknowledges a transaction once the shard positions in its dependency frontier are durable.

The benefit of dependency-closed acknowledgment is not that a transaction has many dependencies. Its benefit appears when a transaction does *not* depend on some slow or lagging WAL shards: global-prefix acknowledgment still waits for those shards, whereas dependency-frontier acknowledgment does not.

Figure 5 reports the result. On YCSB-A the contrast is sharp and holds across the entire sweep: the global-prefix variant stays pinned near the 65,536 backpressure limit with a 131 ms p99 acknowledgment latency at every worker count, whereas the dependency-frontier variant keeps fewer than 500 pending durable commits at every worker count—and far below the global-prefix queue everywhere—with a 1–4 ms p99. At 48 workers, for instance, global-prefix holds 64,197 pending commits and 131 ms while Ayame holds 455 and 2 ms. Even without injecting an artificial straggler, natural progress variation among the WAL shards is enough to keep the global durable prefix behind, so the global-prefix queue never drains; the dependency-frontier rule does not wait for unrelated shards and therefore drains regardless of worker count. On YCSB-B, whose update load is an order of magnitude lighter, the dependency-frontier rule again holds the pending count under about 100 and the p99 at 1 ms across the whole sweep, while the global-prefix rule drains at low-to-mid worker counts but saturates beyond 48 workers as shard progress fluctuates. (Pending counts here are the measurement-window snapshot, as in Table 2; these acknowledgment-policy runs are separate from the main worker-scaling runs, so values differ slightly due to

Table 2 End-to-end YCSB results at 48 transaction worker threads.

Workload	System	Ack tps	p99 μs	Pending	Tx/sync	WAL atomic/tx
YCSB-A	Single WAL	8.8K	8,192	0	1.0	6.01
YCSB-A	P-WAL	49.8K	1,024	0	1.0	6.00
YCSB-A	Ayame	252.7K	2,048	318	25.4	0.00
YCSB-B	Single WAL	25.7K	4,096	0	2.5	0.90
YCSB-B	P-WAL	165.5K	512	0	2.5	0.90
YCSB-B	Ayame	466.9K	512	83	16.7	0.00
YCSB-C	Single WAL	871.6K	53	0	0.0	0.00
YCSB-C	P-WAL	807.5K	58	0	0.0	0.00
YCSB-C	Ayame	811.7K	57	0	0.0	0.00

Ack tps is durable-acknowledgment throughput; Tx/sync is commits made durable per `fdatasync`; WAL atomic/tx is per-transaction WAL-side global-counter accesses. Ayame uses 9 flusher threads and 1 committer at 48 workers.

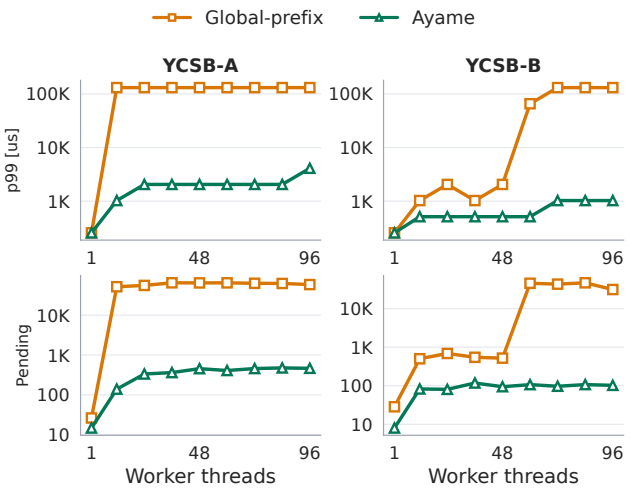

Fig. 5 Acknowledgment-policy comparison (top: p99 latency; bottom: pending durable commits). Both variants use the same asynchronous WAL pipeline and differ only in the acknowledgment condition. The global-prefix rule saturates on YCSB-A across the whole sweep, while the dependency-frontier rule stays bounded at every worker count.

run-to-run variation.) This comparison also decomposes the end-to-end gain. Moving from P-WAL's worker-wait commit to Ayame's asynchronous worker/flusher/committer pipeline (techniques **A1** and **A2**), while still acknowledging at the global durable prefix, raises YCSB-A throughput at 48 workers from 50K to 444K durable acknowledgments per second—about 8.9×, and the source of Ayame's throughput advantage over P-WAL. Switching the acknowledgment rule of that same pipeline from global-prefix to the dependency frontier (**A3**) is *not* a throughput optimization in this short measurement window: it lowers throughput to 251K. Its role is to prevent the global-prefix rule from accumulating a large durable-ack backlog and long tail acknowledgment latency, cutting pending durable commits from 64,197 to 455 and the p99 from 131 ms to 2 ms. Because the two variants share the same pipeline and differ only in the acknowledgment condition, this comparison is itself a complete ablation of **A3**, and no separate ablation experiment is required.

We next ask whether this benefit requires dense dependency frontiers. Zipfian skew does not answer this question: it changes which keys are hot but not which WAL shards wrote them, so in a non-partitioned workload the writers of the accessed versions stay spread across all shards and the dependency frontier remains dense. We therefore vary the frontier width directly with an abort-free partitioned workload in which each worker owns a key partition and a remote-access probability controls how often a transaction touches other partitions. With `remote` = 0 every transaction depends only on its local shard—a sparse frontier of one shard per transaction (nz/tx = 1)—and with remote accesses the frontier quickly becomes dense (about nine shards). The jump is abrupt because dependency frontiers compose transitively—once a transaction touches a remote partition it inherits that partition's own frontier—so even a 1% remote-access probability pulls in almost all shards. Because this abort-free workload commits—and therefore logs—on every transaction, it is the most write-intensive workload in our evaluation; the flush group of 256 commits (Table 1) keeps the flush pipeline unsaturated for *both* acknowledgment policies, so the remaining difference reflects only the acknowledgment rule rather than flush-pipeline saturation.

Table 3 reports the result. The global-prefix rule essentially saturates the pending-ack queue and holds a 131 ms p99 at every density, including the fully sparse case, because it waits for the global durable prefix regardless of what a transaction actually depends on. The dependency-frontier rule keeps the pending count to a few hundred commits and the p99 to about 2 ms across the whole range; its throughput is again slightly lower, as in the comparison. Crucially, even when each transaction depends on a single shard (nz/tx = 1), the dependency-frontier rule still keeps pending durable commits and tail latency far below the global-prefix rule: about 400 versus 65,000 pending commits, and 2 ms versus 131 ms p99. The value of dependency-closed acknowledgment is therefore not that a transaction has many dependencies, but that it does not wait for the shards it does not depend on, and this value remains when frontiers are sparse.

4.5 Hardware Counters and Overhead

Table 4 reports a perf-stat comparison of the three systems at 48 worker threads, and three effects explain Ayame's throughput. First, Ayame amortizes storage synchronization: it makes 23 commits durable per `fdatasync` on YCSB-A and 14 on YCSB-B, whereas Single WAL and P-WAL synchronize roughly once per transaction (Tx/sync of 1.0–2.5).

Table 3 Dependency-density experiment at 48 worker threads on an abort-free partitioned workload. The remote-access probability sets the frontier width (nz/tx is the mean number of shards per dependency frontier).

Remote	nz/tx	Policy	Ack tps	p99 ms	Pending
0%	1.0	Global	438.9K	131	64,626
0%	1.0	Ayame	261.7K	2	397
1%	8.9	Global	437.3K	131	63,618
1%	8.9	Ayame	256.8K	2	325
10%	8.9	Global	436.6K	131	64,602
10%	8.9	Ayame	252.8K	2	358
100%	9.0	Global	438.9K	131	64,924
100%	9.0	Ayame	238.5K	2	349

Table 4 Perf-stat summary at 48 transaction worker threads. Cores is CPU-core utilization, Cyc/tx is CPU cycles per committed transaction, Ctx sw. is total context switches, and Tx/sync is commits per `fdatasync`.

Workload	System	Ack tps	Cores	Cyc/tx	Ctx sw	Tx/sync
YCSB-A	Single WAL	8.1K	1.8	552K	133K	1.0
YCSB-A	P-WAL	48.7K	22.1	1.15M	1.48M	1.0
YCSB-A	Ayame	237.4K	23.5	243K	4.21M	23.0
YCSB-B	Single WAL	22.1K	1.4	165K	140K	2.5
YCSB-B	P-WAL	162.6K	18.0	282K	1.29M	2.5
YCSB-B	Ayame	414.4K	24.3	145K	4.12M	14.4
YCSB-C	Single WAL	680.6K	48.4	184K	4K	0.0
YCSB-C	P-WAL	685.5K	48.3	183K	4K	0.0
YCSB-C	Ayame	683.0K	48.4	184K	34K	0.0

Second, Ayame spends far fewer CPU cycles per committed transaction than P-WAL (243K versus 1.15M on YCSB-A): in P-WAL a worker cannot proceed until its own commit is durable, so worker progress is coupled to storage synchronization, whereas Ayame's workers enqueue a log record and return to execution. Single WAL, in contrast, barely uses the machine (1.8 cores) because its workers serialize on a single shared log. Third, this decoupling is paid for in context switches, of which Ayame performs the most, reflecting the worker/flusher/committer hand-offs. On read-only YCSB-C the three systems converge—within a few percent of one another on 48 cores with near-identical counters—confirming that these differences come from the durable-logging path rather than from transaction execution. These perf-stat runs are separate from the main throughput runs and carry profiling overhead, so their absolute throughputs are lower (for instance, YCSB-C reads about 683K here versus 812K in Table 2); we use them only for the relative cross-system comparison.

5. Related Work

6. Conclusion

This paper presented Ayame, a timestamp-integrated, dependency-closed parallel WAL protocol for SSN-based main-memory transaction processing. Ayame reuses SSN's commit timestamp as the logical LSN (**A1**), separates transaction execution from durable acknowledgment across worker, flusher, and committer roles (**A2**), and acknowledges each transaction once its own log record and its dependency frontier are durable (**A3**). The first two techniques deliver the throughput—a 4.1- to 6.5-fold improvement over P-WAL on YCSB-A across the worker sweep—while the third bounds the durable-acknowledgment backlog and tail latency without waiting for unrelated WAL shards, which we proved recoverable (Lemma 1, Theorem 1).

The scope of this paper is the durable-acknowledgment path and its safety: we evaluated durable-acknowledgment throughput, pending durable commits, and tail latency, and we established that an acknowledged transaction is always recoverable together with the transactions it depends on. We did not measure crash-recovery time or replay throughput; quantifying recovery cost, and evaluating Ayame under a wider range of workloads and storage devices, are left as future work.

Acknowledgments

References

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[3] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[4] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[5] Ports, D. R. K. and Grittner, K.: Serializable Snapshot Isolation in PostgreSQL, *Proc. VLDB Endow.*, Vol. 5, No. 12, pp. 1850–1861 (online), DOI: 10.14778/2367502.2367523 (2012).

[6] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Efficiently Making (Almost) Any Concurrency Control Mechanism Serializable, *The VLDB Journal*, Vol. 26, No. 4, pp. 537–562 (online), DOI: 10.1007/s00778-017-0463-8 (2017).

[7] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[8] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[10] Zheng, W., Tu, S., Kohler, E. and Liskov, B.: Fast Databases with Fast Durability and Recovery Through Multicore Parallelism, *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '14, USENIX Association, pp. 465–477 (online), available from ⟨https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_wenting⟩ (2014).

[11] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Scalable Logging Through Emerging Non-Volatile Memory, *Proc. VLDB Endow.*, Vol. 7, No. 10, pp. 865–876 (online), available from ⟨https://www.vldb.org/pvldb/vol7/p865-wang.pdf⟩ (2014).

[12] 神谷, 孝., 川島, 英., 星野, 喬., 建部, 修., Komei, K., Hideyuki, K., Takashi, H. and Osamu, T.: P-WAL: Paralell Write-ahead Logging, **情報処理学会論文誌データベース（*TOD*）**, Vol. 10, No. 1, pp. 24–39 (online), available from ⟨https://cir.nii.ac.jp/crid/1050282812885062144⟩ (2017).

[13] Kawashima, H.: Next-generation Database Tsurugi: Con-

currency Control and Recovery Systems, *IPSJ Magazine*, Vol. 66, No. 4, pp. e9–e15 (online), DOI: 10.20729/0002001646 (2025).

[14] Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R. and Sears, R.: Benchmarking cloud serving systems with YCSB, *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, New York, NY, USA, Association for Computing Machinery, p. 143–154 (online), DOI: 10.1145/1807128.1807152 (2010).